

Chapter 5: Real-time Communication

Welcome back to the CinéConnect tutorial! In [Chapter 4: Third-Party Movie Data \(TMDB\)](#), we explored how our application efficiently fetches a vast amount of movie information from [TMDB](#). While this is great for displaying static content like movie details, what if we want people to talk to each other instantly, or see new movie comments appear without refreshing the page?

Instant Conversations: Why Real-time Communication Matters

Imagine you're in a live discussion about a movie with friends on CinéConnect, or watching a new comment pop up on a movie page. If the app had to constantly "ask" the server, "Is there a new message? Is there a new comment?", it would be like having a conversation where you have to say "Hello? Are you there? Anything new?" every second. This "polling" approach is very inefficient and makes the app feel slow.

This is where "Real-time Communication" comes in. It's about enabling instant, two-way conversations between users and the server. Instead of constantly asking for updates, the server can immediately "push" new information to all connected users the moment it happens.

Our main goal in this chapter is to understand how CinéConnect uses [Socket.io](#) to create persistent, two-way connections that allow for instant updates and interactive experiences like live chat rooms and dynamic comments.

Let's dive into this "sophisticated walkie-talkie system"!

The Instant Connection: [Socket.io](#)

Think of [Socket.io](#) as a highly advanced walkie-talkie system for your application.

1. **Traditional HTTP (like in previous chapters):** Each request (like fetching a movie list) is a separate, one-off question and answer. You ask, the server answers, then the connection closes.
2. **[Socket.io](#) (Real-time Communication):** When you connect with [Socket.io](#), it establishes a *persistent*, open line of communication – like keeping your walkie-talkie button pressed for a continuous conversation. This "two-way connection" means both your app (the client) and the server can send messages to each other at any time, instantly.

This continuous connection is perfect for:

- **Live Chat:** When you send a message, it immediately appears for everyone else in the chat room.
- **Live Comments:** A new comment on a movie instantly shows up for anyone viewing that movie page.
- **Notifications:** Instant alerts when something important happens.

Use Case: Sending and Receiving Live Chat Messages

Let's walk through the most common real-time use case: participating in a live chat room on CinéConnect.

The User Experience

1. You open the "Global Chat" room. You see a history of recent messages.
2. Another user sends a message, and it instantly appears in your chat window.
3. You type a message and hit "Send".

4. Your message immediately appears in your chat window and for everyone else in the room.

How to Use Live Chat in the Frontend (`useChatRoom`)

In CinéConnect, the `ChatPage` uses a special React hook called `useChatRoom` to manage the chat experience. This hook handles fetching chat history and sending/receiving messages in real-time.

```
// apps/frontend/src/routes/chat.tsx (Simplified)
import { useState, useRef, useEffect } from "react";
import { useChatRoom } from "@/hooks/useChatRoom"; // Our real-time chat tool
import { Send } from "lucide-react";

function ChatPage() {
  const [selectedRoomId, setSelectedRoomId] = useState("global");
  const [inputValue, setInputValue] = useState("");
  const messagesEndRef = useRef<HTMLDivElement>(null);

  // This hook gives us everything needed for the chat room!
  const { messages, isLoading, isError, error, sendMessage, isSending } =
    useChatRoom({ roomId: selectedRoomId });

  // Scroll to the bottom of the chat when new messages arrive
  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages.length]);

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    const text = inputValue.trim();
    if (!text || isSending) return;
    sendMessage(text); // Send the message using the hook!
    setInputValue(""); // Clear the input field
  };

  return (
    <div>
      {/* ... (UI for room selection and messages display) ... */}
      <div className="space-y-3">
        {messages.map((m, index) => (
          <div key={m.id ?? `msg-${index}-${m.createdAt}`}>
            <span>{m.senderName}</span>
            <p>{m.content}</p>
          </div>
        ))}
        <div ref={messagesEndRef} />
      </div>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
          placeholder="Écrivez un message..."
        />
      </form>
    </div>
  );
}
```

```
        />
        <button type="submit" disabled={!inputValue.trim() || isSending}>
          <Send /> Envoyer
        </button>
      </form>
    </div>
  );
}
```

In this simplified example:

- `useChatRoom({ roomId: selectedRoomId })` is the core. It provides:
 - `messages`: An array of chat messages, which updates automatically in real-time.
 - `isLoading`, `isError`, `error`: Status for fetching initial message history.
 - `sendMessage`: A function to send a new message.
 - `isSending`: A flag to indicate if a message is currently being sent.
- When a new message arrives (either from history or real-time), the `messages` array updates, causing the component to re-render and display the new message.
- When you call `sendMessage(text)`, the `useChatRoom` hook handles sending that message through `Socket.io`.

Under the Hood: `Socket.io` in Action

Let's trace how a message gets from one user's keyboard to everyone else's screen using `Socket.io`.

Diagram: Message Flow with `Socket.io`

1. **Send Message**: When you (the "Sender Frontend") type a message and click "Send," `useChatRoom` calls a function in the `Frontend Socket Library` (our client-side `Socket.io` setup).
2. **Emit Event**: The `Frontend Socket Library` uses the open `Socket.io` connection to "emit" a `message` event to the `Backend Socket Server`, containing the chat room ID and the message text.
3. **Receive and Persist**: The `Backend Socket Server` receives this `message` event. It then tells the `Backend Message Service` to save this new message into the `Database`.
4. **Broadcast**: After the message is saved (or even if saving temporarily fails, to ensure real-time feel), the `Backend Socket Server` "broadcasts" a `message` event to *all* connected users who are currently "listening" to that specific chat room.
5. **Receive and Display**: The `Frontend Socket Library` on every "Receiver Frontend" (including the sender's own browser) receives this broadcasted `message` event. It then updates the `messages` state in the `useChatRoom` hook, causing the chat UI to instantly display the new message.

Deeper Dive: Connecting and Talking

Let's look at the key parts of the code that make this happen.

1. Connecting to the `Socket.io` Server (Frontend: `socket.ts`)

First, our frontend needs to establish that persistent connection.

```
// apps/frontend/src/lib/socket.ts (Simplified)
import { io, type Socket } from "socket.io-client";
import { socketHttpOrigin } from "../api-origin";

let sharedSocket: Socket | null = null;

export function getSocket(): Socket {
  if (!sharedSocket) {
    const origin = socketHttpOrigin();
    sharedSocket = io(origin, {
      path: "/socket.io",
      withCredentials: true, // IMPORTANT: Sends cookies (like JWT)
      autoConnect: true,
      reconnection: true, // Automatically tries to reconnect if connection drops
      reconnectionAttempts: Number.POSITIVE_INFINITY,
      reconnectionDelay: 1000,
      reconnectionDelayMax: 30_000,
    });
    // ... (logic to re-join rooms after reconnect) ...
  }
  return sharedSocket;
}

// ... (joinSocketRoom, leaveSocketRoom functions) ...
```

- `getSocket()`: This function creates or returns a single shared `Socket.io` client instance.
- `io(origin, { ... })`: This creates the `Socket.io` connection.
- `withCredentials: true`: This is crucial! It tells the browser to include `httpOnly cookies` (like our `JWT` from [Chapter 2: User Authentication](#)) when connecting. This allows the backend `Socket.io` server to identify the user.
- `reconnection: true`: This ensures that if the network connection temporarily drops, `Socket.io` will automatically try to reconnect, providing a robust real-time experience.

2. Joining and Leaving Chat Rooms (Frontend: `socket.ts` and `useChatRoom.ts`)

Chat messages are typically sent within "rooms." Users can join specific rooms (e.g., "global" chat, or `film:123` for a movie discussion).

```
// apps/frontend/src/lib/socket.ts (Simplified)
const roomRefCounts = new Map<string, number>(); // Tracks how many components
want a room open

export function joinSocketRoom(roomId: string): void {
  const id = roomId.trim();
  if (!id) return;
  const next = (roomRefCounts.get(id) ?? 0) + 1;
  roomRefCounts.set(id, next);
  if (next === 1) { // Only emit "join_room" if this is the first component
    joining
  }
}
```

```

    getSocket().emit("join_room", id);
  }
}

export function leaveSocketRoom(roomId: string): void {
  const id = roomId.trim();
  if (!id) return;
  const prev = roomRefCounts.get(id) ?? 0;
  if (prev <= 0) return;
  const next = prev - 1;
  if (next <= 0) { // Only emit "leave_room" if this is the last component leaving
    roomRefCounts.delete(id);
    if (sharedSocket) {
      sharedSocket.emit("leave_room", id);
    }
  } else {
    roomRefCounts.set(id, next);
  }
}

```

- `joinSocketRoom` and `leaveSocketRoom`: These functions manage subscriptions to chat rooms. They use a "reference counter" (`roomRefCounts`) so that multiple components (e.g., if a movie page and a chat page both want `film:123` messages) can safely subscribe and unsubscribe without closing the room too early. The actual `join_room` and `leave_room` events are emitted to the `Backend Socket Server` only when the first component joins or the last component leaves.

3. Receiving Messages and Sending (Frontend: `useChatRoom.ts`)

The `useChatRoom` hook puts everything together for the client.

```

// apps/frontend/src/hooks/useChatRoom.ts (Simplified)
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { useEffect, useState } from "react";
import { messageService, type ChatMessage } from "@service/message.service";
import { getSocket, joinSocketRoom, leaveSocketRoom } from "@lib/socket";

export function useChatRoom(options: UseChatRoomOptions): UseChatRoomReturn {
  const { roomId, limit = 50 } = options;
  const queryClient = useQueryClient();
  const [liveMessages, setLiveMessages] = useState<ChatMessage[]>([]); // For
  // real-time updates

  // 1. Fetch initial message history (using React Query, like in Chapter 3)
  const { data: historyData, isLoading, isError, error, refetch } = useQuery({
    queryKey: ["messages", roomId],
    queryFn: () => messageService.getByRoom({ room: roomId!, limit }),
    enabled: roomId != null && roomId.length > 0,
  });

  // 2. Handle sending messages via Socket.io
  const sendMutation = useMutation({

```

```

    mutationFn: (text: string) => {
      // Emit the "message" event to the backend Socket.io server
      getSocket().emit("message", { roomId, text });
      return Promise.resolve(); // This is optimistic, message might fail on
server
    },
    onSuccess: () => {
      // After sending, invalidate React Query cache to fetch updated history
      if (roomId) queryClient.invalidateQueries({ queryKey: ["messages", roomId]
});
    },
  });

// 3. Set up Socket.io event listeners
useEffect(() => {
  if (roomId == null || roomId === "") return;
  const socket = getSocket();
  joinSocketRoom(roomId); // Join the selected room on the backend

  // Listener for initial message history when joining a room
  const onHistory = (payload: { roomId: string; messages: ChatMessage[] }) => {
    if (payload.roomId === roomId) {
      setLiveMessages([...payload.messages].reverse()); // Update state with
history
    }
  };

  // Listener for new real-time messages
  const onMessage = (payload: { roomId: string; text: string; senderName?:
string }) => {
    if (payload.roomId === roomId) {
      // Add new message to current live messages
      setLiveMessages((prev) => [...prev, {
        id: undefined, roomId, content: payload.text,
        senderName: payload.senderName ?? "Anonyme", createdAt: new
Date().toISOString()
      }]);
    }
  };

  socket.on("message_history", onHistory);
  socket.on("message", onMessage);

  return () => { // Cleanup when component unmounts or roomId changes
    socket.off("message_history", onHistory);
    socket.off("message", onMessage);
    leaveSocketRoom(roomId); // Leave the room on the backend
  };
}, [roomId]); // Re-run effect if roomId changes

// Prioritize live messages, otherwise use history from API
const messages = liveMessages.length > 0 ? liveMessages : (historyData?.messages
?? []).reverse();

```

```
return { messages, isLoading, isError, error, sendMessage: sendMutation.mutate,
  isSending: sendMutation.isPending };
}
```

- **useQuery**: Fetches the initial chat history using traditional HTTP (leveraging **React Query**'s caching benefits from [Chapter 3: Frontend Data & State Management](#)).
- **sendMutation**: When **sendMessage** is called, it triggers **sendMutation.mutate**, which emits the **message** event to the **Backend Socket Server** using **getSocket().emit()**.
- **useEffect**: This is the heart of the real-time functionality.
 - It calls **joinSocketRoom(roomId)** to tell the backend **Socket.io** server that this user wants to receive messages for this room.
 - It attaches **socket.on("message_history", onHistory)** and **socket.on("message", onMessage)** listeners. These functions are called *automatically* by **Socket.io** whenever the backend sends a **message_history** or **message** event for the current room.
 - When **onHistory** or **onMessage** are triggered, they update the **liveMessages** state, which causes the UI to instantly update.
 - The **return** function in **useEffect** cleans up the listeners and calls **leaveSocketRoom(roomId)** when the component is no longer active, ensuring efficient resource use.

4. The Backend **Socket.io** Server (**socket.ts**)

On the backend, a dedicated **Socket.io** server listens for connections and events.

```
// apps/backend/src/socket.ts (Simplified)
import { Server as HttpServer } from 'http';
import { Server, Socket } from 'socket.io';
import { parse as parseCookieHeader } from 'cookie';
import jwt from 'jsonwebtoken';
import { getJwtSecret } from './config';
import { COOKIE_ACCESS } from './utils/authCookies';
import { MessageService, chatSenderDisplayName } from
  './services/message.service';
import { sanitizeUserText } from './utils/sanitize';

export function createSocketServer(httpServer: HttpServer): Server {
  const io = new Server(httpServer, {
    cors: { origin: true, credentials: true }, // Allow frontend connections
    path: '/socket.io',
  });

  // Middleware to authenticate users on connection
  io.use(async (socket, next) => {
    const cookieHeader = socket.handshake.headers.cookie;
    const cookies = cookieHeader ? parseCookieHeader(cookieHeader) : {};
    const token = cookies[COOKIE_ACCESS]; // Get JWT from httpOnly cookie

    if (token) {
      try {
        const decoded = jwt.verify(token, getJwtSecret()) as { userId: number };

```

```

        socket.data.userId = decoded.userId; // Attach user ID to the socket
        // ... (fetch user name from DB and attach to socket.data.senderName) ...
    } catch {
        // Token invalid, user remains anonymous
    }
}
next(); // Continue connection
});

io.on('connection', (socket: Socket) => {
    // Event listener for client joining a room
    socket.on('join_room', async (roomId: string) => {
        socket.join(roomId); // Add socket to the Socket.io room
        io.to(roomId).emit('user_joined', { roomId, userId: socket.data.userId });
    // Notify others
        // Send message history to the newly joined client
        try {
            const { messages: history } = await MessageService.getByRoom(roomId, 50,
0);
            socket.emit('message_history', { roomId, messages: history });
        } catch (e) { /* ignore DB errors */ }
    });

    // Event listener for client sending a message
    socket.on('message', async (payload: { roomId?: string; text?: string }) => {
        const { roomId, text: rawText } = payload;
        if (!roomId || !rawText) return;
        const cleanText = sanitizeUserText(rawText);
        if (!cleanText) return;

        // Persist message to database
        let createdAt = new Date().toISOString();
        try {
            const row = await MessageService.create(socket.data.userId ?? null,
roomId, cleanText);
            if (row?.createdAt) createdAt = row.createdAt.toISOString();
        } catch (e) { /* persist failed; still broadcast */ }

        // Broadcast the message to all clients in the room
        io.to(roomId).emit('message', {
            roomId, text: cleanText, timestamp: createdAt,
            userId: socket.data.userId, senderName: socket.data.senderName
        });
    });

    // ... (other event listeners like 'leave_room') ...
});

return io;
}

```


- `io.use(async (socket, next) => { ... })`: This is a `Socket.io` middleware that runs *before* a client fully connects. It's used for authentication. It tries to extract the `JWT` from `httpOnly cookies` (or `socket.handshake.auth`) and verify it. If valid, `socket.data.userId` and `socket.data.senderName` are set, making user information available for all subsequent events from this socket. This links directly to our authentication system from [Chapter 2: User Authentication](#).
- `io.on('connection', (socket: Socket) => { ... })`: This listener fires every time a new client connects to the `Socket.io` server. Inside, we define specific event listeners for that client's `socket`.
- `socket.on('join_room', async (roomId: string) => { ... })`: When a client emits a `join_room` event, the server calls `socket.join(roomId)`, which adds that specific client's `socket` to an internal `Socket.io` "room." Now, any messages broadcast to `roomId` will be sent to this client. It also fetches initial `message_history` from the database using `MessageService` and sends it *only* to the joining client (`socket.emit`).
- `socket.on('message', async (payload: { ... }) => { ... })`: When a client emits a `message` event, the server:
 1. Sanitizes the message text (`sanitizeUserText`).
 2. Calls `MessageService.create` to save the message in the `Database`.
 3. Uses `io.to(roomId).emit('message', message)` to broadcast the message to *all* clients currently in that specific `roomId`. This is the core of real-time communication!

5. Persisting Messages (Backend: `message.service.ts`)

The `Backend Message Service` is responsible for interacting with our database to store and retrieve chat messages.

```
// apps/backend/src/services/message.service.ts (Simplified)
import { db } from '../db';
import { messages, users } from '../db/schema';
import { desc, eq, sql } from 'drizzle-orm';
import { sanitizeUserText } from '../utils';

export const MessageService = {
  /** Persist a chat message. */
  async create(senderId: number | null, roomId: string, content: string) {
    const clean = sanitizeUserText(content);
    if (!clean) return null;
    const [row] = await db
      .insert(messages)
      .values({ senderId, roomId, content: clean })
      .returning(); // Returns the created row, including createdAt timestamp
    return row;
  },

  /** Get paginated message history for a room, newest first. */
  async getByRoom(roomId: string, limit = 50, offset = 0) {
    const rows = await db
      .select({
        id: messages.id, content: messages.content, createdAt: messages.createdAt,
        senderId: messages.senderId, senderName: users.name, // Join to get
        sender's name
      })
      .from(messages)
      .innerJoin(users, eq(messages.senderId, users.id))
      .orderBy(desc(messages.createdAt))
      .limit(limit)
      .offset(offset);
    return rows;
  }
};
```

```
    })
    .from(messages)
    .leftJoin(users, eq(messages.senderId, users.id))
    .where(eq(messages.roomId, roomId))
    .orderBy(desc(messages.createdAt))
    .limit(limit)
    .offset(offset);

    // ... (logic to get total count) ...

    return { messages: rows, total: rows.length, limit, offset };
  },
};
```

- `MessageService.create`: This function inserts a new chat message into the `messages` table in our Database.
- `MessageService.getByRoom`: This function retrieves a paginated list of messages for a given `roomId`, which is used to populate the initial chat history when a user joins a room. It also joins with the `users` table to fetch the `senderName` so we can display who sent the message.

Socket.io vs. React Query: Different Tools for Different Jobs

It's important to remember that `Socket.io` for real-time communication and `React Query` (from [Chapter 3: Frontend Data & State Management](#)) for data fetching solve different problems:

Feature	Socket.io (Real-time Communication)	React Query (Data Fetching & Caching)
Primary Use	Instant, two-way, push-based communication.	Request-response, pull-based data fetching.
Connection	Persistent, open connection.	Short-lived, request-specific connections.
Data Flow	Server pushes updates to clients; clients push messages to server.	Clients ask for data; server responds.
Key Benefits	Instant updates, live interactions, reduced polling.	Caching, background re-fetching, loading states, error handling, automatic retries for static data.
Analogy	Live walkie-talkie conversation.	Ordering food from a menu.
Example Use	Chat messages, live comments, instant notifications.	Movie lists, user profile details, search results.

In CinéConnect, `useChatRoom` cleverly combines both: it uses `React Query` to fetch the *initial historical messages* efficiently, and then switches to `Socket.io` for *live, real-time updates* once the connection is established.

Conclusion

In this chapter, we've explored the exciting world of real-time communication in CinéConnect. We learned how `Socket.io` establishes persistent, two-way connections, acting like a sophisticated walkie-talkie to enable instant message exchange. We saw how the `useChatRoom` hook simplifies building live chat features in the frontend, and we peered behind the scenes to understand how the `Backend Socket Server` authenticates users, manages chat rooms, persists messages to the database, and broadcasts updates to all connected clients. This technology is essential for dynamic, interactive experiences within our application.

Now that we understand how information is instantly shared, in the next chapter, we'll dive deeper into the structure and principles of our server-side logic in [Backend API Core](#).